COURSE CODE: PY-2026 V3.11+

Python Programming Syllabus & Coursework

A comprehensive deep dive into Python fundamentals, collections, object-oriented programming, and data analysis.

syllabus_overview.py

```python
import sys
from course import content

def start_learning():
  units = [
    "Fundamentals",
    "Collections",
    "OOP",
    "Data Science"
  ]
  for unit in units:
    print(f"Mastering {unit}...")
```

PRESENTED BY

Dr. Kamakshi Rautela

Agenda

Comprehensive Python Course Structure

6 Units • 115 Slides

UNIT I

Introduction to Python

Syntax, Types, Operators, Control Flow, and Loops.

UNIT II

Collections & Functions

Lists, Tuples, Sets, Dictionaries, Functional Programming.

UNIT III

Modules & Regex

Packages, Standard Libs, Regular Expressions.

UNIT IV

File & Exception Handling

I/O Operations, Context Managers, Error Handling.

UNIT V

Class & Objects

OOP Concepts, Inheritance, Polymorphism, Magic Methods.

UNIT VI

Data Analysis & Viz

NumPy Arrays, Pandas DataFrames, Matplotlib Plotting.

- Master Python Syntax
- Build Robust Applications
- Analyze Data Efficiently
- Visualize Insights

GETTING STARTED

How to Use This Deck

Modern Standards

Examples use **Python 3.11+** syntax. Code is shown in `monospace` with explicit outputs.

Active Learning

Look for **Try It Yourself** exercises. Pause the slide and run the code in your IDE (VS Code or PyCharm).

Checkpoints

Each unit concludes with a **Quiz**. Use these to validate your understanding before proceeding to the next unit.

Slide Icon Legend

CODE	Executable snippet
EXERCISE	Practice task
ALERT	Warning / Pitfall

Tip: Keep a Python REPL open!

SECTION 1 OF 6

Introduction to Python

- Python Setup & Interactive Mode
- Basic Syntax, Tokens & Comments
- Variables & Data Types
- Operators & Expressions
- Control Flow: Conditionals & Loops

01

```
def main():
    print("Starting Unit I...")
    x = 10
    if x > 5:
        print("Learning Python")

    for i in range(5):
        # Basic
        pro

    whil
```

UNIT I: INTRODUCTION

What is Python? Why Python?

Simple & Expressive

A high-level, interpreted language designed for **readability**. Its syntax mimics English, making it ideal for rapid prototyping and learning.

General Purpose Powerhouse

From **Web Development** (Django, Flask) to **Data Science** (Pandas), **AI** (PyTorch), and Automation. It runs everywhere (Windows, macOS, Linux).

Batteries Included

Includes a comprehensive **Standard Library** and a vast ecosystem of 300,000+ packages on PyPI to solve almost any problem.

Try it in REPL

Check your version and start the shell:

```
$ python --version
Python 3.11.4
$ python
Type "help", "copyright"...
>>> _
```

Tip: The ">>>" prompt means Python is listening!

Working with Python: Setup

Installation

Install the latest version of Python (3.11+) from `python.org`. Ensure you check "Add Python to PATH" during installation on Windows.

Virtual Environments

Always create a virtual environment (`venv`) for each project to manage dependencies in isolation and avoid conflicts.

IDEs

Recommended editors: **VS Code** (with Python extension) or **PyCharm** for a full-featured experience.

terminal

```
# Check installation version
$ python --version

# Create a virtual environment (.venv)
$ python -m venv .venv

# Activate environment (Windows)
$ .venv\Scripts\activate

# Activate environment (Mac/Linux)
$ source .venv/bin/activate
```

OUTPUT / RESULT

```
Python 3.11.4
(.venv) $ _
```

Interactive vs Scripting Mode

CORE CONCEPT

hello_script.py

```python
# 1. Script Mode: Save commands in a file
def greet_user(name):
    """Simple greeting function"""
    return f"Hello, {name}! Welcome to Python."
# Main execution block
if __name__ == "__main__":
    user = "Developer"
    message = greet_user(user)
    print(message)
```

TERMINAL OUTPUT

```
$ python hello_script.py
Hello, Developer! Welcome to Python.
$ _
```

Key Differences

- **Interactive (REPL):** Use for instant feedback, testing small snippets, or as a calculator (`>>>` prompt).
- **Script Mode (.py):** Use for writing permanent programs, defining functions, and building applications.

Dynamic Typing

Binding vs. Declaration

Python variables are not explicitly declared (no `int x`). A variable is simply a name bound to an object. The type belongs to the object, not the variable.

Dynamic Reassignment

You can rebind the same variable name to objects of different types during execution. This flexibility is a hallmark of dynamic typing.

Common Pitfall

Because types change, performing operations (like `+`) without verifying types can lead to runtime `TypeError`.

dynamic_typing.py

```python
# 1. x bound to an integer object
x = 10
print(f"x is {x}, type: {type(x)}")

# 2. x rebound to a string object
x = "Python"
print(f"x is {x}, type: {type(x)}")

# 3. Pitfall: Runtime Error
# y = x + 5 # TypeError: can't add str + int
```

OUTPUT / RESULT

```
x is 10, type: <class 'int'>
x is Python, type: <class 'str'>
```

Mutable vs Immutable Types

CRITICAL CONCEPT

FEATURE	MUTABLE	IMMUTABLE
Definition	Object state **can be changed** after it is created.	Object state **cannot be changed** once created.
Data Types	`list` `dict` `set` `bytearray`	`int` `float` `bool` `str` `tuple`
Memory Behavior	Operations modify the object **in-place** (same memory address/ID).	Operations create a **new object** at a different memory address.
Use Case	Use for collections that need frequent updates (add/remove items).	Use for constants, dictionary keys, or ensuring data integrity.

Why It Matters

Understanding mutability is crucial for avoiding bugs when passing arguments to functions (pass-by-reference behavior) and when using objects as dictionary keys (must be hashable/immutable).

```python
# Immutable (String)
s = "hello"; old_id = id(s)
s = s + " world" # New object created
print(id(s) == old_id) # False

# Mutable (List)
lst = [1, 2]; old_id = id(lst)
lst.append(3) # Modified in-place
```

UNIT I: INTRODUCTION

Basic Syntax & Indentation

Indentation Defines Blocks

Unlike C++ or Java which use `{}`, Python uses **whitespace** to define scope. All statements in a block must align perfectly.

PEP 8 Style Guide

The official style guide recommends using **4 spaces** per indentation level. Avoid mixing tabs and spaces to prevent `TabError`.

Common Pitfalls

`IndentationError` occurs if alignment is off by even one space. Use an IDE that visualizes indentation guides.

Syntax Quick Reference

: — **Colon**
Starts a new indented block (functions, loops, if).

\n — **Newline**
Ends a statement. Semicolons ; are optional and rarely used.

— **Comments**
Everything after # is ignored by the interpreter.

Pro Tip: Configure your editor to convert Tab to Spaces automatically.

Comments and Docstrings

BEST PRACTICES

documentation_demo.py

```python
# This is a block comment explaining the logic
def calculate_area(radius):
    """
    Calculates the area of a circle.
    Args: radius (float)
    Returns: Area (float)
    """
    pi = 3.14159 # Inline comment for pi
    return pi * (radius ** 2)
# View docstring via help() or __doc__
print(calculate_area.__doc__)
```

TERMINAL OUTPUT

```
$ python documentation_demo.py
    Calculates the area of a circle.

    Args: radius (float)
    Returns: Area (float)
$ _
```

Comments vs. Docstrings

- **Comments (#):** Ignored by Python. Use them to explain "why" complex logic exists for developers.
- **Docstrings ("""):** Retained at runtime. Used for API documentation and displayed by `help()`.

Strings: Values and Operators

Initialization

Enclose text in single `' '`, double `" "`, or triple quotes `'''` `'''` (for multiline).

Operators

Concatenate strings using `+`, repeat them with `*`, and check existence with `in` (membership).

Indexing & Slicing

Access characters via zero-based index `[i]` or extract substrings using slicing `[start:stop]`.

string_ops.py

```python
# String initialization
a = 'Hello'
b = "World"

# Operators: + (concat), * (repeat)
print(a + " " + b) # Concatenation
print(a * 3) # Repetition
print('He' in a) # Membership

# Indexing and Slicing
```

OUTPUT / RESULT

```
Hello World
HelloHelloHello
True
P
tho
```

Strings: Values and Operators

Initialization

Enclose text in single `' '`, double `" "`, or triple quotes `''' '''` (for multiline).

Operators

Concatenate strings using `+`, repeat them with `*`, and check existence with `in` (membership).

Indexing & Slicing

Access characters via zero-based index `[i]` or extract substrings using slicing `[start:stop]`.

string_ops.py

```python
# Operators: + (concat), * (repeat)
print(a + " " + b) # Concatenation
print(a * 3) # Repetition
print('He' in a) # Membership

# Indexing and Slicing
text = "Python"
print(text[0]) # Index 0: 'P'
print(text[2:5]) # Slice: 'tho'
```

OUTPUT / RESULT

```
Hello World
HelloHelloHello
True
P
tho
```

String Methods and Formatting

UNIT I

str_methods.py

```python
text = " Python Programming "
print(text.strip()) # Removes leading/trailing spaces
print(text.lower()) # Converts to lowercase
# Split and Join
data = "apple,banana,cherry"
fruits = data.split(",") # Returns a list
print(fruits)
print(" | ".join(fruits)) # Joins list elements
# String Formatting
ver = 3.11
print("Python {} is great".format(ver)) # .format() method
print(f"Version: {ver}") # f-string (Best Practice)
```

TERMINAL OUTPUT

```
$ python str_methods.py
Python Programming
python programming
['apple', 'banana', 'cherry']
apple | banana | cherry
Python 3.11 is great
Version: 3.11
```

Key Methods

- **Modification:** `strip()`, `lower()`, `upper()`, `replace()` create new strings (strings are immutable).
- **Formatting:** Prefer **f-strings** (Python 3.6+) over `.format()` or `%` for readability and performance.

UNIT I: FUNDAMENTALS

Numeric Data Types

Integers (`int`)

Arbitrary precision integers. They can be as large as memory allows, unlike C/Java fixed-size integers.

Ex: `10`, `-5`, `10**100`

Floating Point (`float`)

Implemented using double precision (64-bit IEEE 754). Accurate up to ~15 decimal digits.

Ex: `3.14`, `1.2e-3`

Complex & Boolean

Complex: Real + Imaginary (`3 + 4j`).

Bool: Subtype of int. `True (1)` and `False (0)`.

Quick Check & Casting

TYPE CONVERSION

```
int(3.9)          # 3 (Truncates)
float(5)                  # 5.0
bool(0)                 # False
```

Precision Pitfall

```
>>> 0.1 + 0.2
0.30000000000000004
```

Pop Quiz:

What is `type(10 / 2)`?

Answer: float (5.0)

Input & Output + Escape Sequences

I/O BASICS

io_demo.py

```python
# 1. Input: Reads user data as string
name = input("Enter name: ")
# 2. Output: Custom separators & endings
print("Hi", name, sep=" | ", end=" -> ")
print("Welcome!")
# 3. Escape Sequences: \n, \t, \\, \"
print("Path:\tC:\\Users\\Admin")
print("Quote:\t\"Python is fun\"\nDone.")
# 4. f-strings (formatted output)
age = 25
print(f"User {name} is {age} years old.")
```

TERMINAL OUTPUT

```
$ python io_demo.py
Enter name: Alice
Hi | Alice -> Welcome!
Path: C:\Users\Admin
Quote: "Python is fun"
Done.
User Alice is 25 years old.
```

Quick Notes

- **input():** Always returns a string. Use `int()` or `float()` to convert types if needed.
- **sep & end:** `sep=' '` is default (space). `end='\n'` is default (new line).
- **Escapes:** `\n` (Newline), `\t` (Tab), `\\` (Backslash), `\"` (Double Quote).

PYTHON BASICS

Tokens & Identifiers

Python Tokens

The smallest meaningful units in a program.

Categories: Keywords, Identifiers, Literals, Operators, Punctuators.

Identifier Rules

Names given to variables, functions, classes.

- Must start with letter (a-z, A-Z) or underscore (_).
- Followed by letters, digits (0-9), or underscores.
- **Case sensitive** (age ≠ Age).
- Cannot be a keyword.

Keywords

Reserved words with special meaning.

`if` `else` `while` `def` `import` `class`

PEP 8 Naming Conventions

Official Style Guide Recommendation

TYPE	FORMAT
Variable	`snake_case`
Function	`snake_case`
Class	`PascalCase`
Constant	`UPPER_CASE`
Private	`_leading_underscore`

Python treats `myVar`, `MyVar`, and `MYVAR` as three completely different variables.

Operators: Types Overview

UNIT I REFERENCE

CATEGORY	SYMBOLS & DESCRIPTION	EXAMPLE
Arithmetic	+ - * / % ** //	10 // 3 *# 3 (Floor Div)*
Relational	== != > < >= <=	5 >= 3 *# True*
Logical	and or not	True and False *# False*
Assignment	= += -= *= /=	x += 5 *# x = x + 5*
Bitwise	& | ^ ~ << >>	5 & 3 *# 1 (0101 & 0011)*
Membership	in not in	'a' in 'apple' *# True*
Identity	is is not	x is None *# Check ID*

Precedence Matters

Operators are evaluated based on precedence (PEMDAS logic). Arithmetic > Relational > Logical. Always use **parentheses ()** to ensure the intended order of execution.

PRECEDENCE EXAMPLE

```
x = 5; y = 10; z = 2
# Without parens: * happens before +
result = x + y * z # 5 + 20 = 25

# With parens:
result = (x + y) * z # 15 * 2 = 30
```

Precedence and Associativity

OPERATORS

precedence_demo.py

```python
# 1. Arithmetic: * has higher precedence than +
res1 = 10 + 5 * 2 # 10 + 10 = 20
print(f"10 + 5 * 2 = {res1}")
# 2. Parentheses () override default order
res2 = (10 + 5) * 2 # 15 * 2 = 30
print(f"(10 + 5) * 2 = {res2}")
# 3. Logical: 'not' > 'and' > 'or'
# True or (False and False) -> True or False
res3 = True or False and False
print(f"True or False and False = {res3}")
# 4. Chained Comparisons (x < y < z)
x = 15
check = 10 < x < 20 # (10 < x) and (x < 20)
print(f"10 < 15 < 20 is {check}")
# 5. Associativity: ** is Right-to-Left
pow_val = 2 ** 3 ** 2 # 2 ** (3 ** 2) = 2 ** 9
print(f"2 ** 3 ** 2 = {pow_val}")
```

TERMINAL OUTPUT

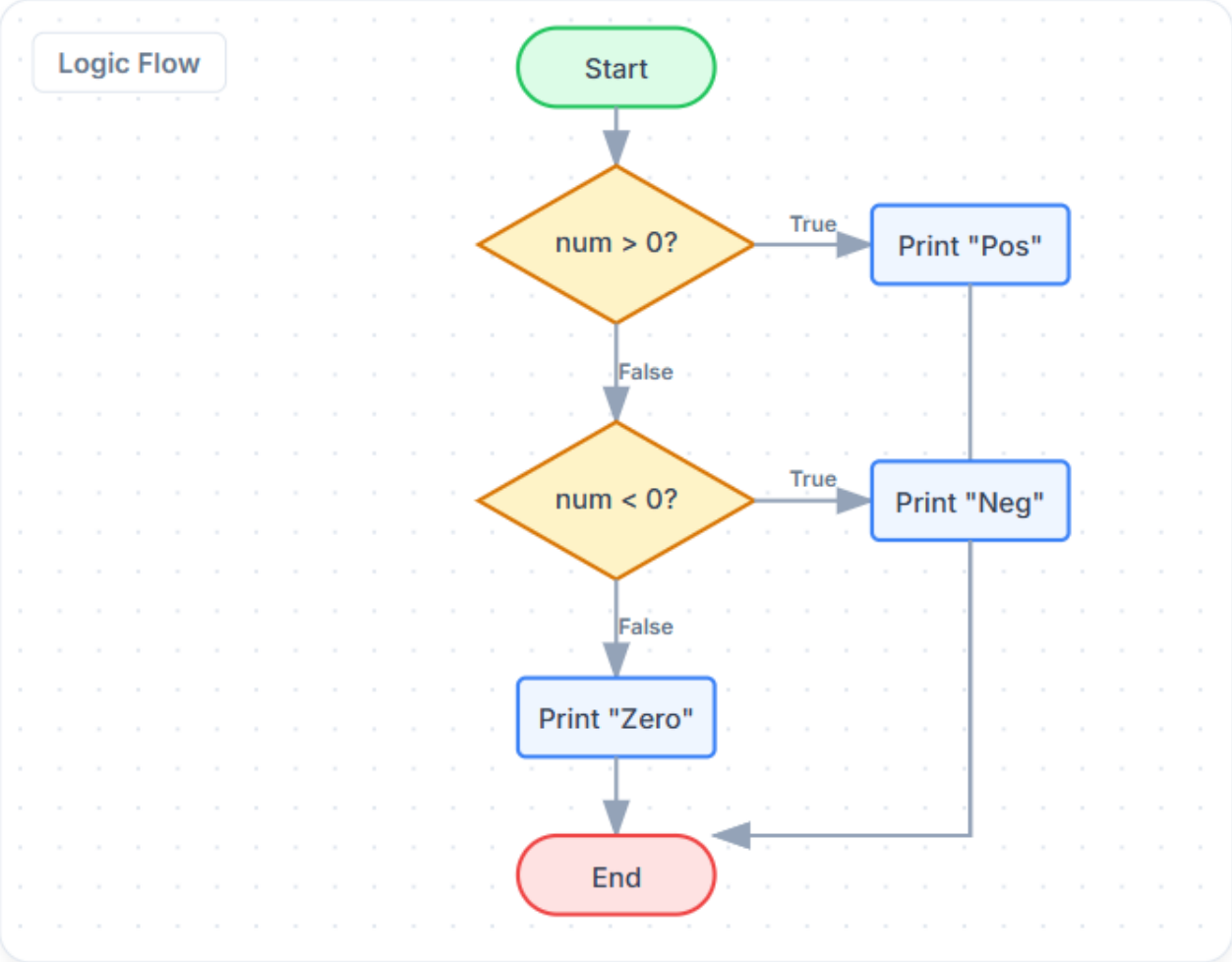
```
$ python precedence.py
10 + 5 * 2 = 20
(10 + 5) * 2 = 30
True or False and False = True
10 < 15 < 20 is True
2 ** 3 ** 2 = 512
$ _
```

Key Rules

- **PEMDAS Logic:** Parentheses `()` > Exponents `**` > Multiplication `*` / Division `/` > Addition `+` / Subtraction `-`.
- **Associativity:** Most operators are **Left-to-Right** (e.g., `10 - 5 - 2` is `(10-5)-2`).
- **Exceptions:** Exponentiation `**` and assignment `=` evaluate **Right-to-Left**.

Decision Making: if / elif / else

CONTROL FLOW



check_number.py

```python
if num > 0:
  print("Positive")
elif num < 0:
  print("Negative")
else:
```

How it executes

1. Python evaluates the `if` condition first. If **True**, it executes the block and skips the rest.
2. If **False**, it moves to the `elif` condition (else if).
3. If all conditions fail, the `else` block executes automatically as a fallback.

Decision Making: match Statement

PYTHON 3.10+

structural_match.py

```python
def http_status(status):
match status:
case 200 | 201: # OR Pattern
return "Success"
case 404:
return "Not Found"
case 418:
return "I'm a teapot"
case _ if status >= 500: # Guard
return "Server Error"
case _: # Wildcard (Default)
return "Unknown Status"
print(http_status(201))
print(http_status(503))
print(http_status(999))
```

TERMINAL OUTPUT

```
$ python structural_match.py
Success
Server Error
Unknown Status
```

Why use match?

- **Structural Pattern Matching:** Not just a switch statement; matches patterns in data structure, types, and values.
- **Syntax:** Uses `match` and `case` blocks. `case _:` acts as the default catch-all.
- **Advanced:** Supports OR patterns (`|`) for multiple values and "guards" (`if` conditions) for logic.

range() and Loop Basics

The range() Function

Generates a sequence of numbers. It is lazy (generates on demand).

- `range(stop)` : 0 to stop-1
- `range(start, stop)` : start to stop-1
- `range(start, stop, step)` : increment by step

The for Loop

Iterates over items of any sequence (e.g., list, string) or iterable object (e.g., range). The `in` keyword is used for iteration.

Iteration Patterns

Use `range()` when you need to iterate a specific number of times or need the index.

loops.py

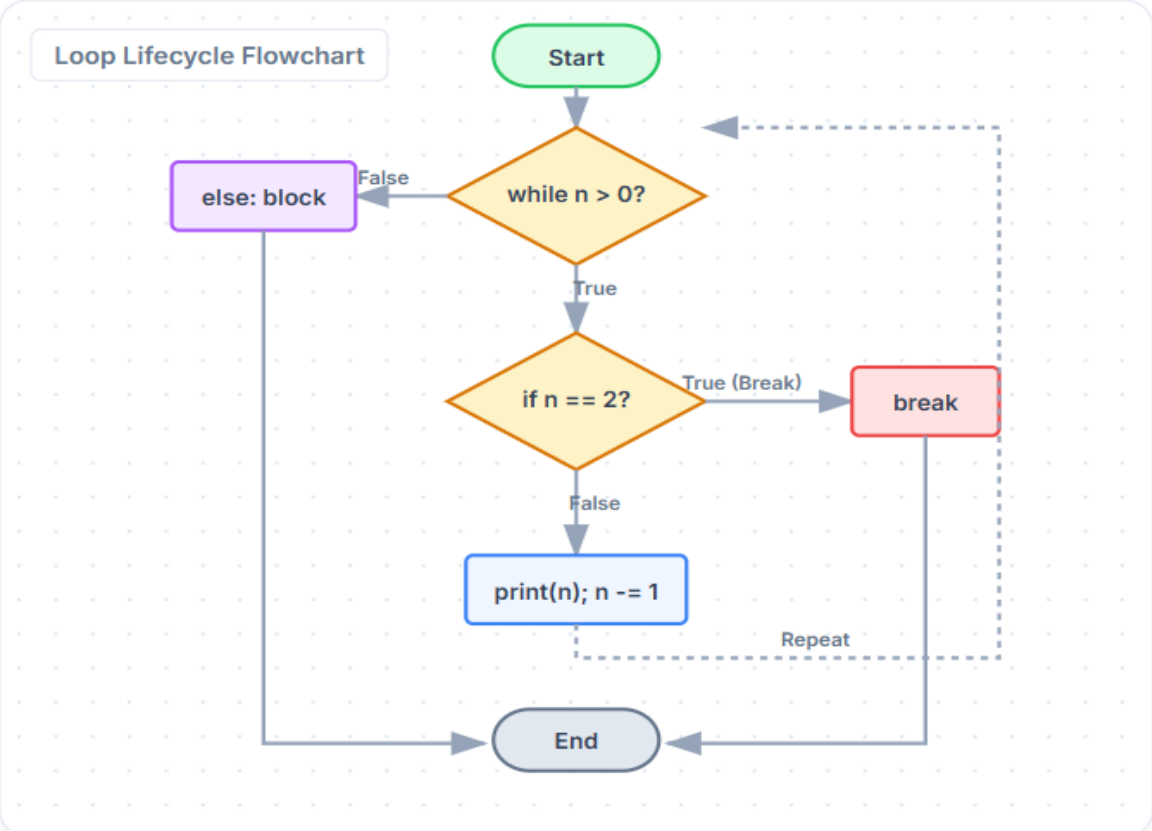
```python
# 1. Basic range: 0 to 4
for i in range(5):
print(i, end=' ')
print() # Newline

# 2. Step argument: 2, 4, 6...
for n in range(2, 9, 2):
print(f"Even: {n}")
```

OUTPUT / RESULT

```
0 1 2 3 4
Even: 2
Even: 4
Even: 6
Even: 8
P-y-t-
```

Loops: while, break, continue & else

CONTROL FLOW



loop_control.py

```python
n = 5
while n > 0:
  if n == 2:
    print("Breaking!")
    break
  print(n)
  n -= 1
else:
  print("Finished normally")
```

Loop Control Statements

1. **while loop**: Executes the block repeatedly as long as the condition `n > 0` evaluates to True.
2. **break**: Terminates the loop *immediately*. Importantly, if `break` hits, the `else` block is **skipped**.
3. **else (in loops)**: A unique Python feature. This block
4. **continue**: Skips the rest of the current iteration code and jumps back to the condition check immediately.

TRY IT YOURSELF

Nested Loops Mini-Exercises

Est. Time: 10 mins

TASK 01

Easy

Multiplication Table

Write a program using nested `for` loops to print a multiplication table for numbers 1 through 5. Format the output so columns align neatly.

- [ ] Done (Output matches example)

TASK 02

Medium

Prime Number Checker

Use a loop to iterate from 2 to 20. For each number, use a nested loop to check if it has any divisors. Print only the prime numbers found.

- [ ] Done (Correctly identifies primes)

EXPECTED OUTPUT (TASK 1)

```
1 2 3 4 5 2 4 6 8 10 3 6 9 12 15 4 8 12 16 20 5
10 15 20 25
```

Hints & Tips

Formatting: Use `print(val, end='\t')` to add a tab without a newline inside the inner loop.

Primes: A number `n` is prime if no number from 2 to `n-1` divides it evenly. You can use a `break` statement if a divisor is found.

Extension Challenge

Modify Task 1 to allow user input for the table size (e.g., up to 10 or 12).

Unit I Checkpoint Quiz

5 Mins

1 Which of the following is an immutable data type in Python?

- A. List
- B. Dictionary
- C. Tuple
- D. Set

2 What is the output of `print(type(10 / 2))`?

- A. <class 'int'>
- B. <class 'float'>
- C. <class 'double'>
- D. <class 'number'>

3 Which operator has the highest precedence?

- A. + (Addition)
- B. and (Logical AND)
- C. ** (Exponentiation)
- D. == (Equality)

4 What keyword is used to skip the current iteration in a loop?

- A. break
- B. pass
- C. skip
- D. continue

Q5. What is printed? PYTHON 3.11

```python
x = 5
if x > 10:
    print("Big")
elif x == 5:
    print("Equal")
else:
    print("Small")
```

- A. Big
- B. Equal
- C. Small